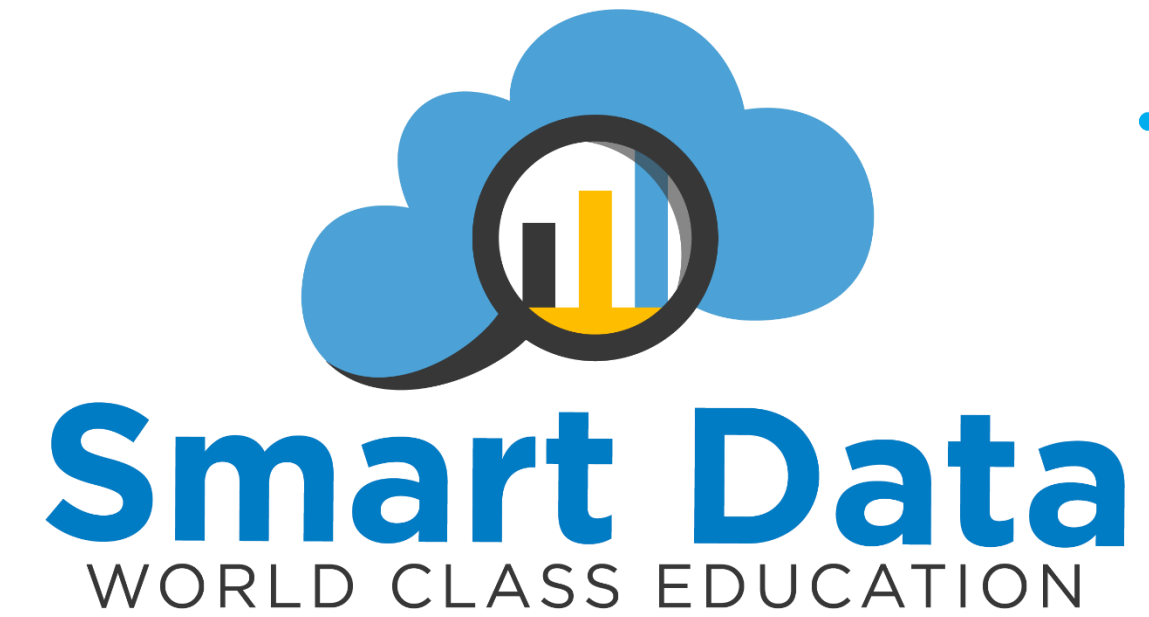
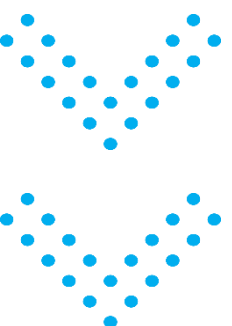
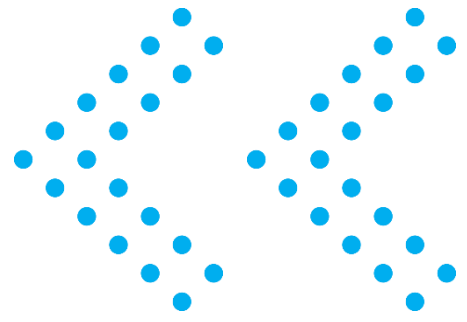
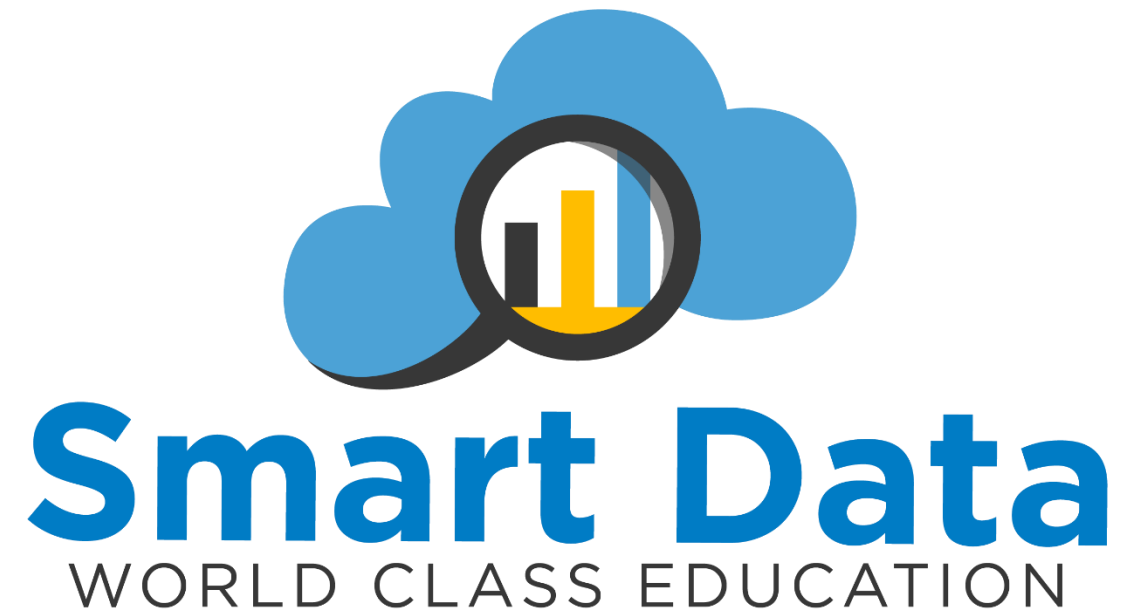




MLOps: Del Modelo al entorno Productivo

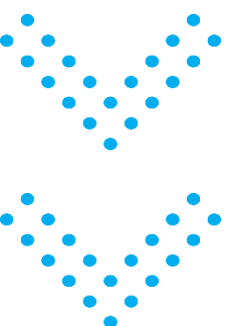
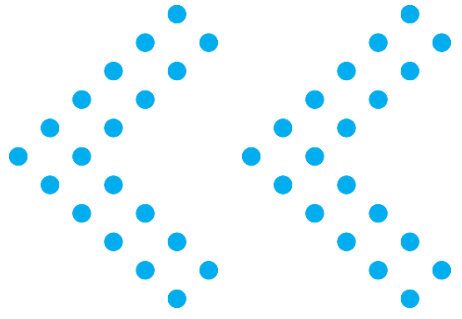




Ajuste y seguimiento de modelos

Instructor:

Julio César Bernal Fernández



Parámetros vs. Hiperparámetros

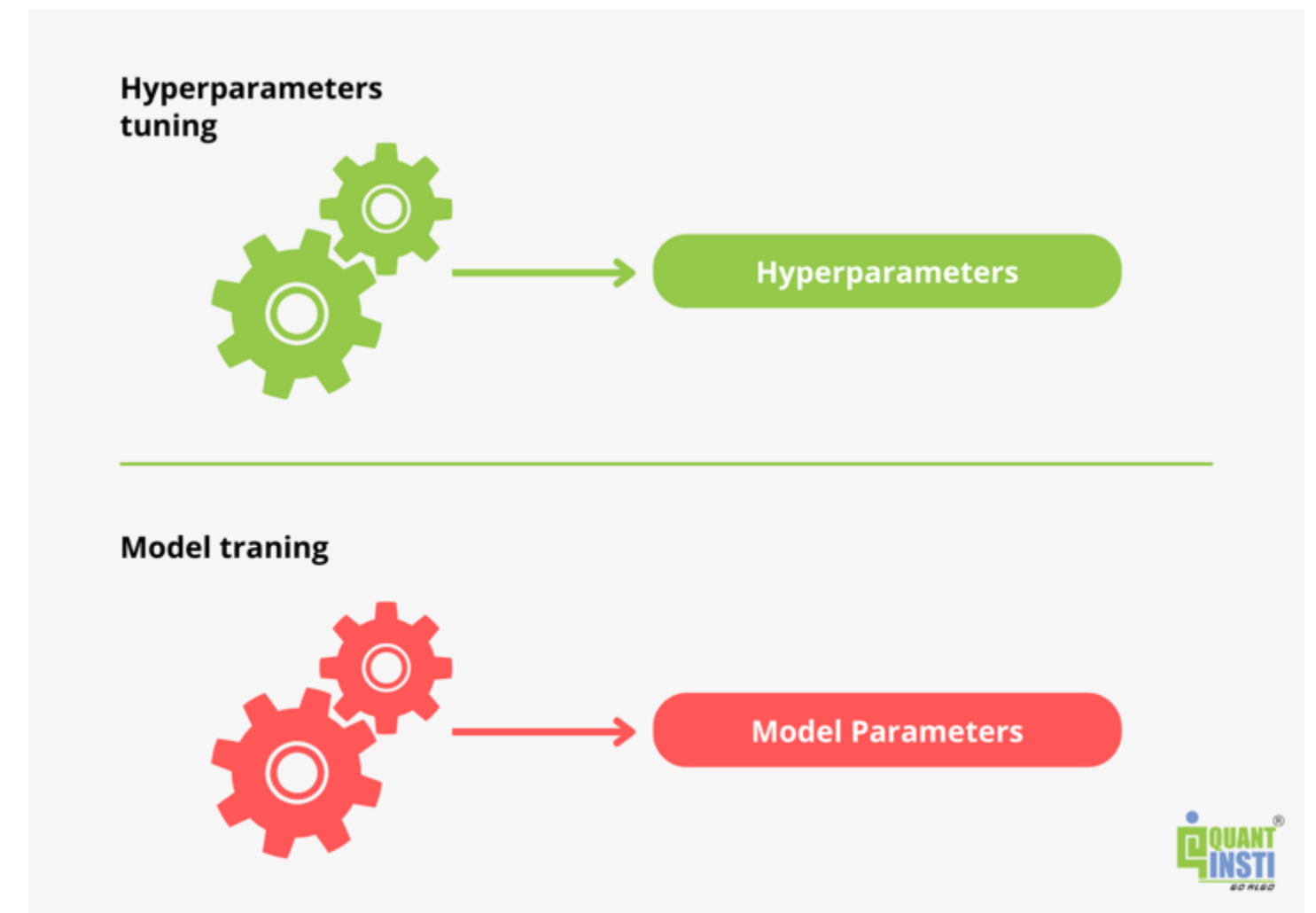
Parámetros del modelo

Son los valores aprendidos automáticamente durante el entrenamiento (e.g., pesos de una red neuronal, coeficientes de regresión).

Hiperparámetros

Son valores de configuración definidos ANTES del entrenamiento. No son aprendidos por el modelo: tasa de aprendizaje (`learning_rate`), profundidad máxima (`max_depth`), número de estimadores (`n_estimators`), regularización (`C`, `alpha`, `lambda`).

Scikit-learn – Parameter estimation: https://scikit-learn.org/stable/modules/grid_search.htm



¿Por qué es Crítico el Tuning?

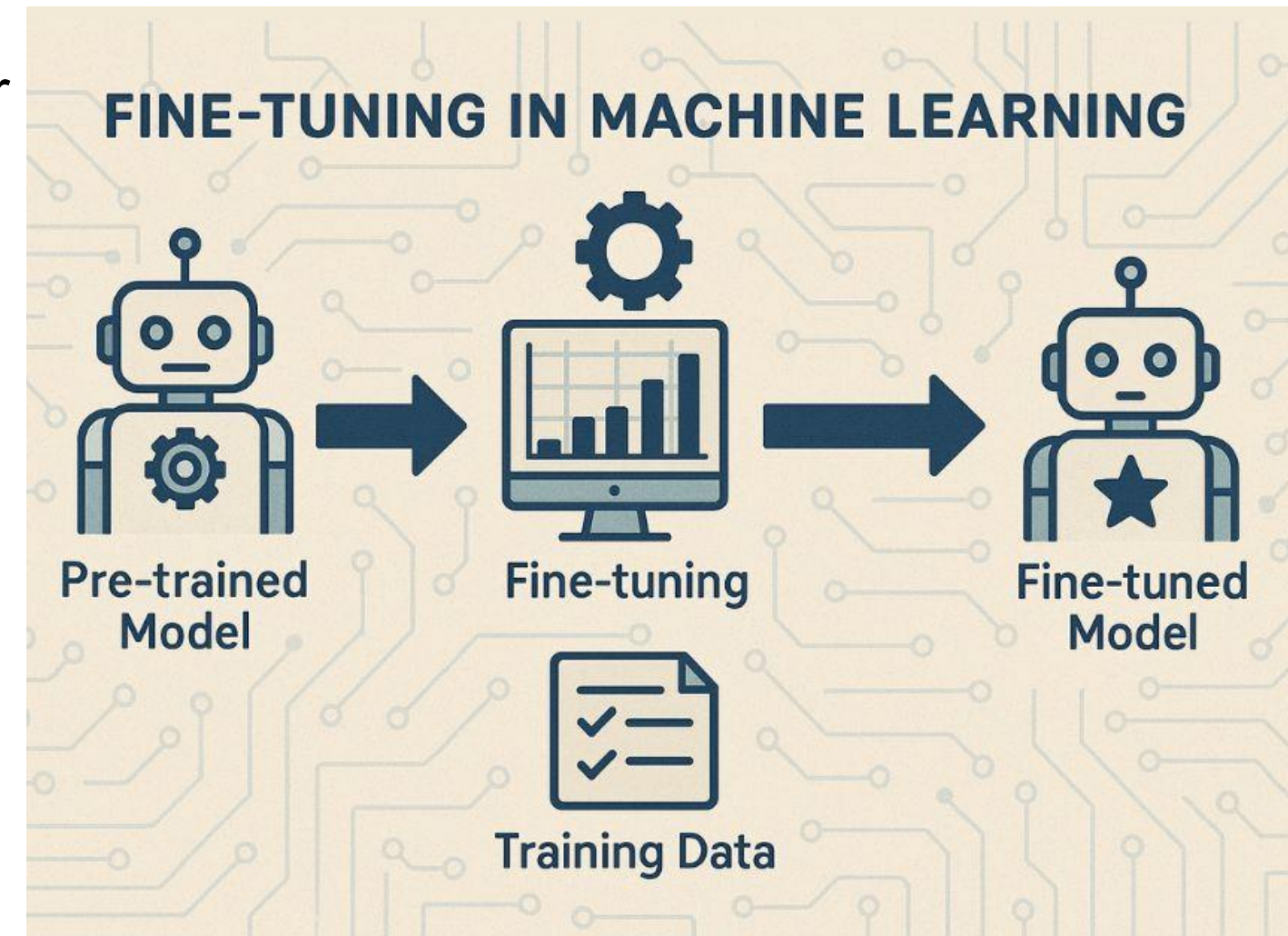
Impacto directo en el rendimiento

Un mismo algoritmo con hiperparámetros distintos puede variar hasta 20–30% en métricas clave (AUC, F1, RMSE). El tuning evita underfitting y overfitting.

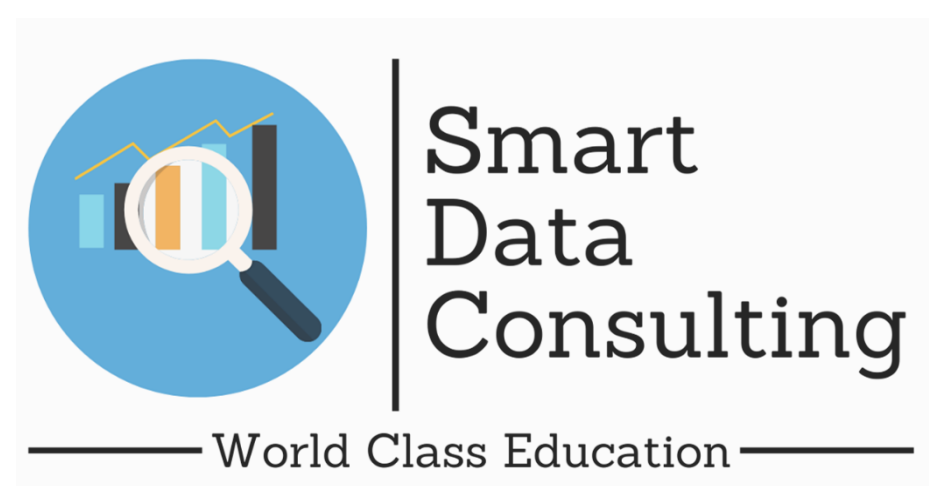
En el ciclo MLOps

El tuning forma parte del pipeline de CI/CD para ML. Cada experimento debe ser rastreable, reproducible y comparable.

Google – Rules of ML (Rule #1): <https://developers.google.com/machine-learning/guides/rules-of-ml>



El Espacio de Búsqueda de Hiperparámetros



Definición

El espacio de búsqueda es el conjunto de combinaciones posibles de hiperparámetros. Puede ser discreto (lista de valores), continuo (rango numérico) o condicional (un hiperparámetro depende de otro).

Ejemplo – Random Forest

- `n_estimators`: [50, 100, 200, 500]
- `max_depth`: [3, 5, 10, None]
- `min_samples_split`: [2, 5, 10]

→ $4 \times 4 \times 3 = 48$ combinaciones posibles (grid completo).

Optuna – Define search spaces:

https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/002_configurations.html

Validación Cruzada en el Proceso de Tuning

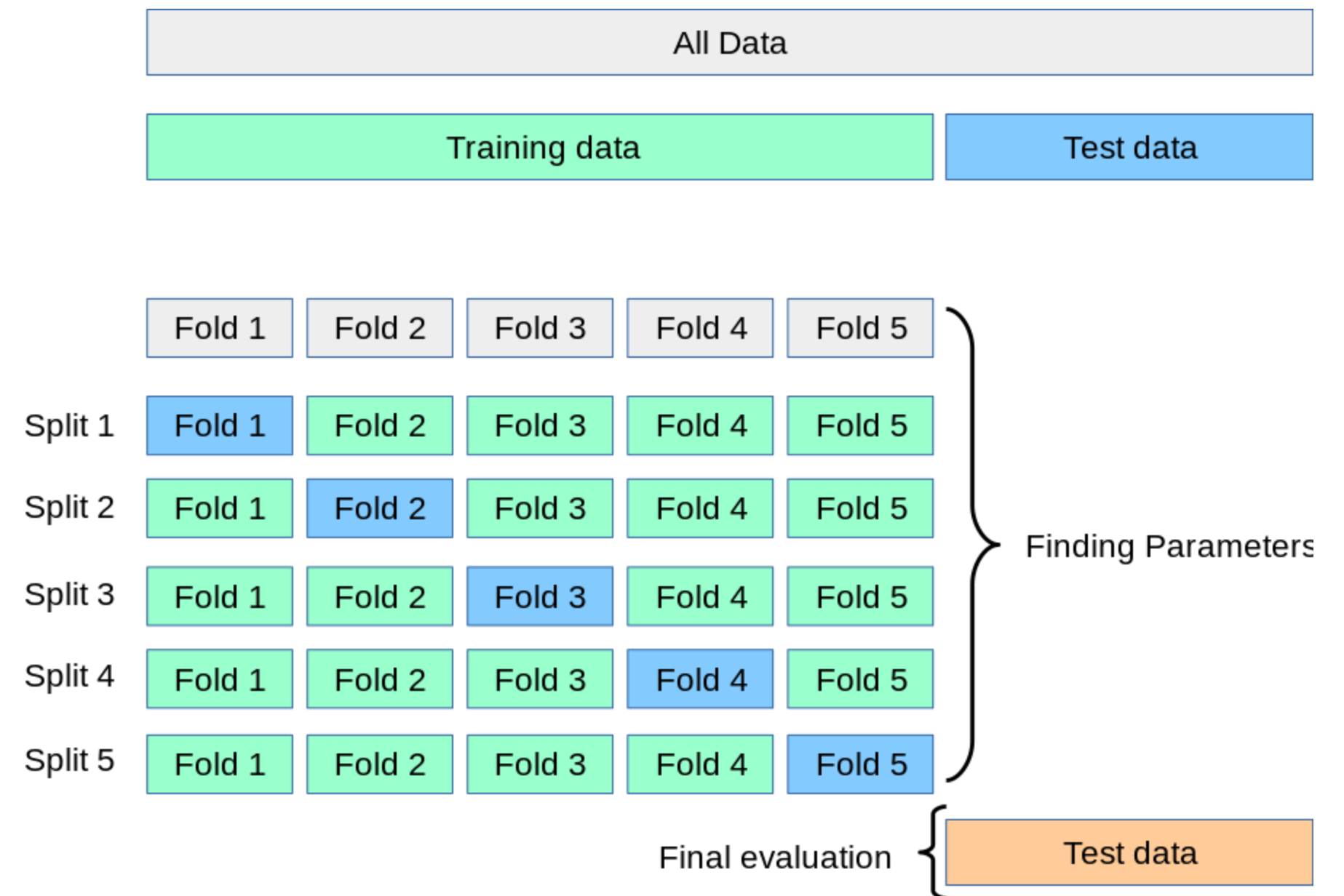
¿Por qué se usa cross-validation?

Para estimar de forma robusta el rendimiento de cada combinación de hiperparámetros sin filtrar información del conjunto de prueba (data leakage).

Estrategias

- K-Fold CV (k=5 o k=10): estándar para datos tabulares.
- Stratified K-Fold: cuando las clases están desbalanceadas.
- TimeSeriesSplit: para datos con dependencia temporal.

Scikit-learn – Cross-validation: https://scikit-learn.org/stable/modules/cross_validation.html



GridSearchCV – Búsqueda Exhaustiva

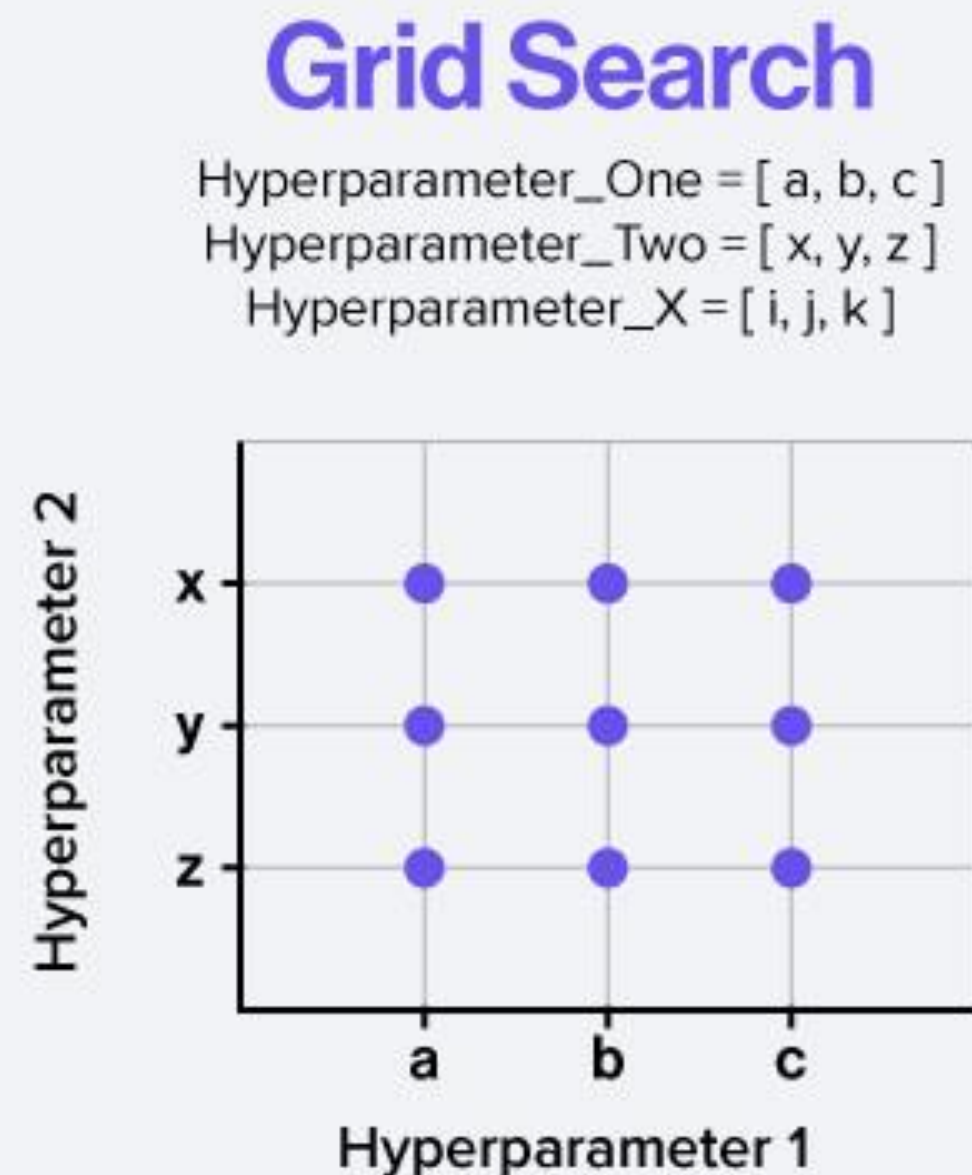
¿Qué hace GridSearchCV?

Prueba TODAS las combinaciones posibles del espacio definido. Garantiza encontrar el óptimo dentro del espacio dado, pero escala pobremente $O(N^k)$.

Implementación básica

- `from sklearn.model_selection import GridSearchCV`
- `param_grid = {'C': [0.1, 1, 10], 'kernel': ['rbf', 'linear']}`
- `gs = GridSearchCV(SVC(), param_grid, cv=5, scoring='f1')`
- `gs.fit(X_train, y_train) → gs.best_params_`

Documentación GridSearchCV: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html



RandomizedSearchCV – Búsqueda Aleatoria

La alternativa eficiente

Muestrea aleatoriamente n_iter combinaciones del espacio de búsqueda. Bergstra & Bengio (2012) demostraron que es más eficiente que grid search para espacios grandes.

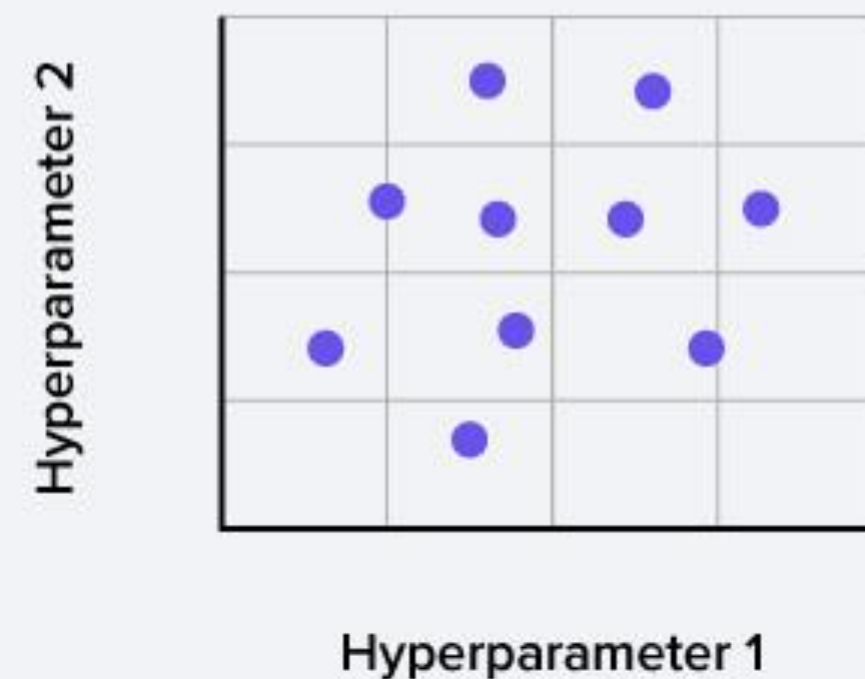
Ventajas clave

- Control de presupuesto: n_iter define el número de evaluaciones.
- Soporta distribuciones continuas (scipy.stats.uniform, loguniform).
- Convergencia práctica en 60–80% de configuraciones.

Paper original: Random Search for Hyper-Parameter Optimization:
<https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf>

Random Search

```
Hyperparameter_One = random.num (range)  
Hyperparameter_Two = random.num (range)  
Hyperparameter_X = random.num (range)
```



Búsqueda Bayesiana – Optimización Inteligente

¿Cómo funciona?

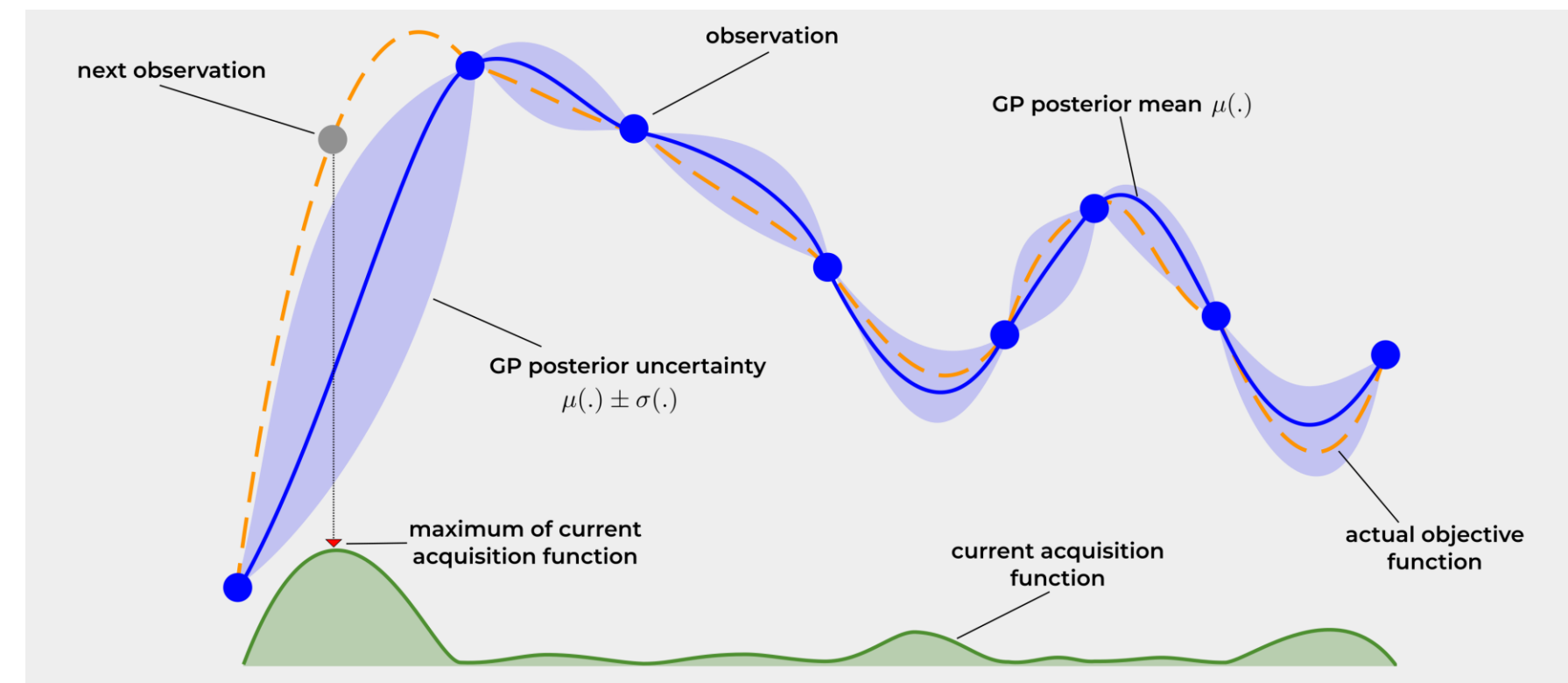
Usa un modelo sustituto (surrogate model, e.g., Gaussian Process o Tree Parzen Estimator) para modelar la función objetivo. Cada evaluación actualiza el modelo y guía la siguiente búsqueda.

Herramientas

- Optuna: TPE por defecto, fácil de integrar con MLflow.
- Hyperopt: pionero en búsqueda bayesiana para ML.
- BayesSearchCV (scikit-optimize): API compatible con sklearn.

Optuna Documentation: <https://optuna.readthedocs.io>

Hyperopt GitHub: <https://github.com/hyperopt/hyperopt>



Hyperband y ASHA – Búsqueda con Presupuesto Adaptativo

Hyperband

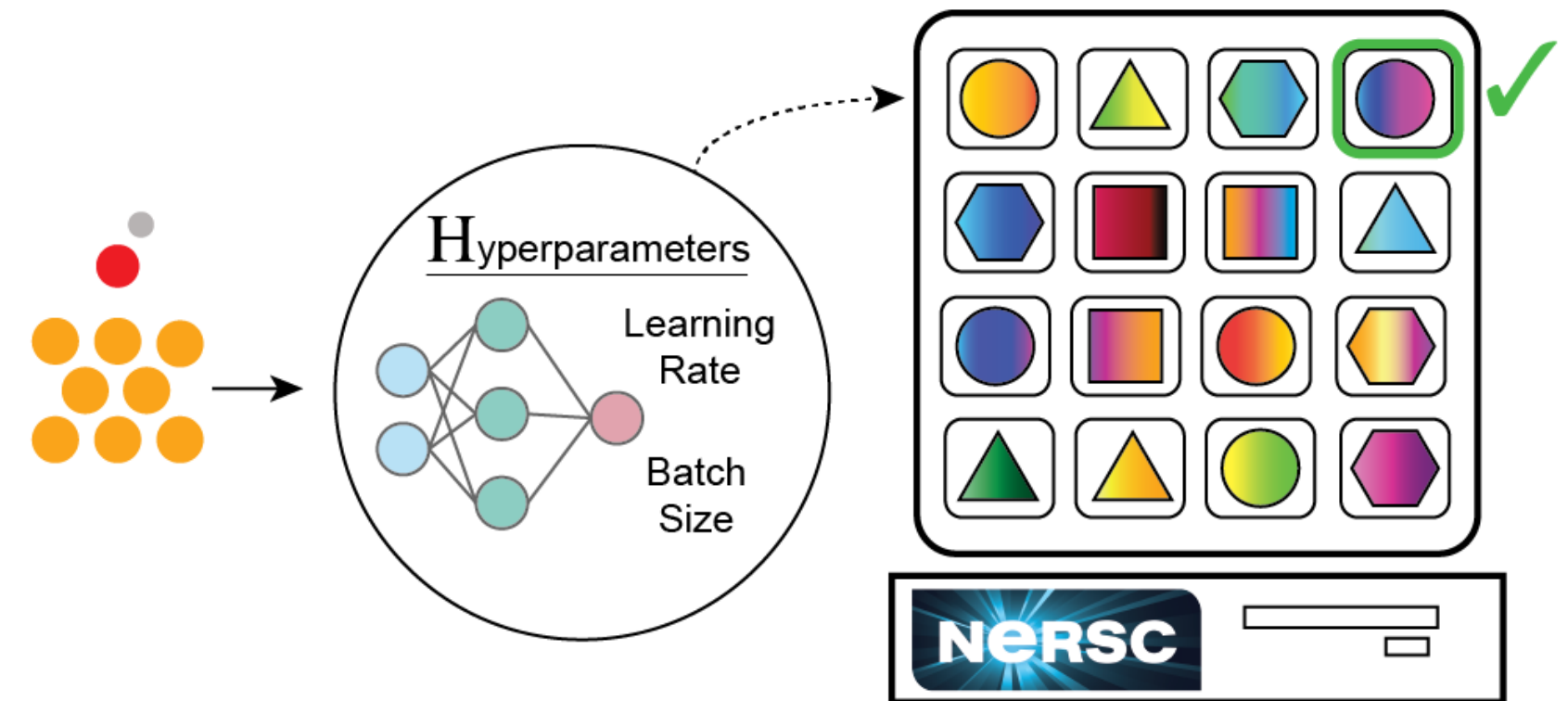
Algoritmo tipo 'torneo': lanza muchas configuraciones con poco presupuesto (epochs, data fraction) y va eliminando las peores. Mucho más rápido que random search puro.

ASHA (Asynchronous Successive Halving)

Versión asíncrona de Hyperband para entornos distribuidos. Implementado en Ray Tune y Optuna Pruners.

Ray Tune – Hyperband: <https://docs.ray.io/en/latest/tune/api/schedulers.html>

Paper Hyperband: <https://arxiv.org/abs/1603.06560>



Comparativa de Métodos de Tuning



Grid Search

✓ Exhaustivo | ✗ Muy lento | Espacio: discreto y pequeño | Uso: prototipos.

Random Search

✓ Rápido | ✓ Soporta espacios continuos | ✗ No usa resultados anteriores | Uso: línea base.

Bayesiano (Optuna/Hyperopt)

✓ Eficiente | ✓ Aprende del historial | ✗ Mayor complejidad | Uso: producción.

Hyperband / ASHA

✓ Muy rápido para DL | ✓ Distribuido | ✗ Requiere métrica intermedia | Uso: redes profundas.

Comparativa detallada: <https://neptune.ai/blog/hyperparameter-tuning-in-python-complete-guide>

MLflow Tracking – Fundamentos

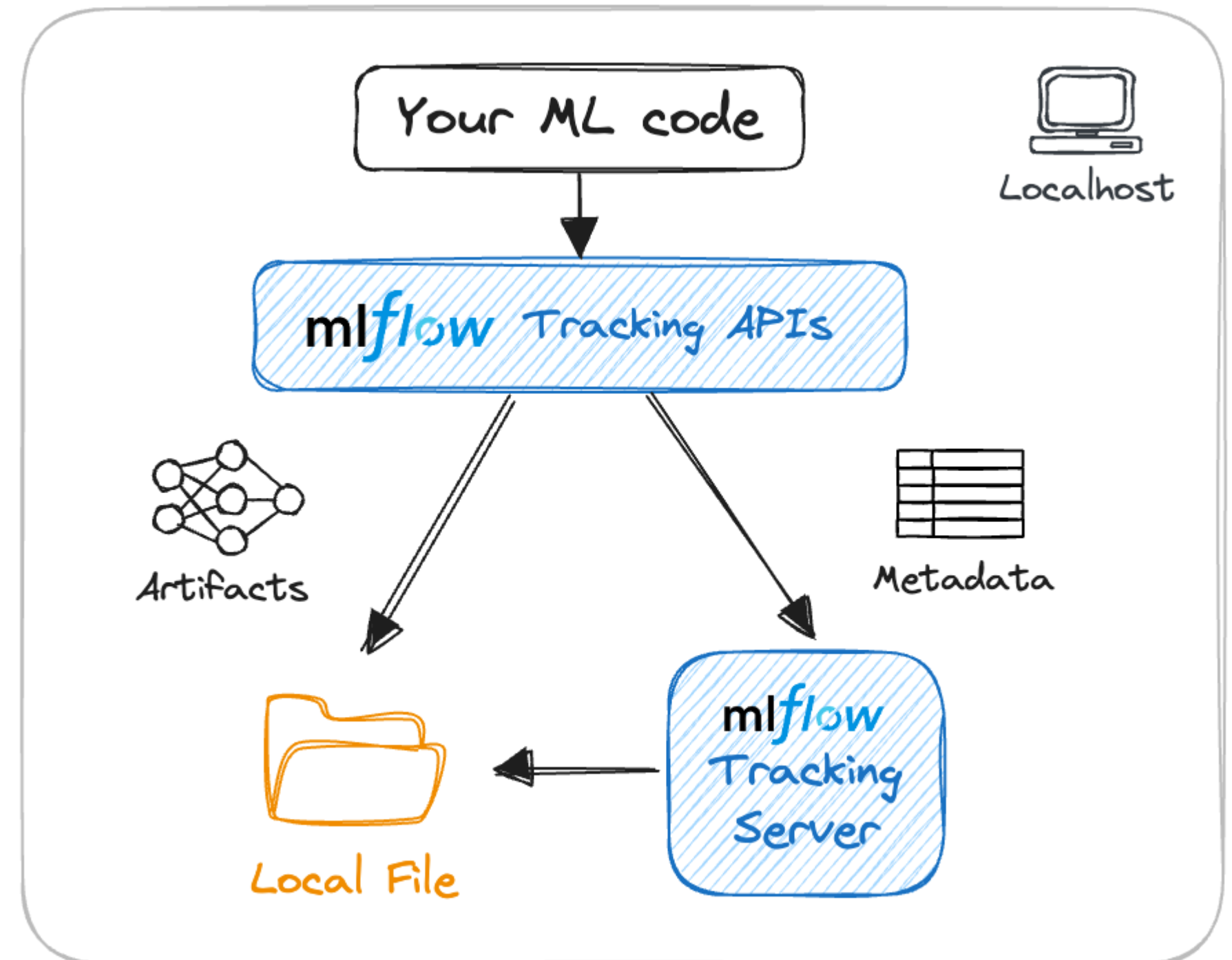
¿Qué es MLflow Tracking?

Componente de MLflow para registrar y comparar experimentos de ML. Almacena parámetros, métricas, artefactos y metadatos de cada ejecución (run).

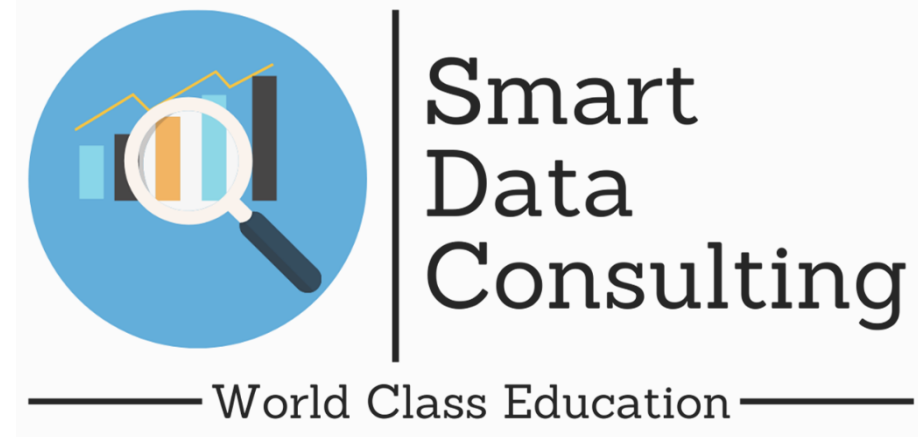
Conceptos clave

- Experiment: agrupación lógica de runs (e.g., 'tuning_random_forest_v1').
- Run: una ejecución con una combinación específica de hiperparámetros.
- Artifact: modelo serializado, plots, datasets.

MLflow Tracking Docs: <https://mlflow.org/docs/latest/tracking.html>



Registrando Hiperparámetros en MLflow



Patrón básico de logging

- `import mlflow`
- `mlflow.set_experiment('tuning_xgboost')`
- `with mlflow.start_run(run_name='run_01'):`
- `mlflow.log_params(best_params)`
- `mlflow.log_metrics({'auc': auc, 'f1': f1})`
- `mlflow.sklearn.log_model(model, 'model')`

Autologging

`mlflow.sklearn.autolog()` registra automáticamente params, métricas y el modelo sin código adicional.

MLflow Autologging: <https://mlflow.org/docs/latest/tracking/autolog.html>

Integración Optuna + MLflow

Patrón de integración

Cada trial de Optuna se mapea a un run de MLflow, creando una trazabilidad completa del proceso de tuning.

- `def objective(trial):`
- `with mlflow.start_run(nested=True):`
- `lr = trial.suggest_float('lr', 1e-4, 1e-1, log=True)`
- `mlflow.log_param('lr', lr)`
- `score = train_model(lr)`
- `mlflow.log_metric('val_auc', score)`
- `return score`

Optuna + MLflow Tutorial:

https://optuna.readthedocs.io/en/stable/tutorial/20_recipes/007_optuna_callback.html

Runs Comparables – Mejores Prácticas



Para que los runs sean comparables

- Misma semilla aleatoria (random_state) en todos los runs.
- Mismo conjunto de datos y mismo split train/val/test.
- Misma métrica objetivo registrada con el mismo nombre.
- Tags para identificar: algoritmo, versión de datos, entorno.

Estandarización con MLflow Projects

MLflow Projects define el entorno (conda.yaml o Docker) garantizando reproducibilidad entre equipos.

MLflow Projects: <https://mlflow.org/docs/latest/projects.html>

MLflow UI – Comparación Visual de Runs



Lanzar la interfaz

Comando: `mlflow ui --port 5000` → `http://localhost:5000`

Funcionalidades clave

- Parallel coordinates plot: visualiza relación hiperparámetros-métrica.
- Comparar N runs simultáneamente en tabla.
- Descargar artefactos (modelos, plots) directamente.
- Filtros por tags, fechas, métricas.

MLflow UI Documentation: <https://mlflow.org/docs/latest/tracking/tracking-server.html>

MLflow con DagsHub (remoto gratuito): https://dagshub.com/docs/integration_guide/mlflow_tracking/

De la Experimentación al Machine Learning Profesional: MLflow + RandomizedSearchCV

Fase 1: Los Engranajes del Modelo (¿Qué ajustamos?)

Hiperparámetros del Algoritmo

Parámetros que definimos manualmente, como la profundidad de un árbol en Random Forest o el tipo de regularización en una regresión logística.



Parámetros del Proceso de Entrenamiento

Factores externos al modelo como el tamaño de la validación, el número de "folds" en validación cruzada y la métrica objetivo (Accuracy, F1, etc.).



Parámetros del Pipeline

Pasos cruciales previos al entrenamiento, incluyendo la normalización de datos, la selección de variables y el balanceo de clases.



Fase 2: La Búsqueda Inteligente con RandomizedSearchCV

Definición del Espacio de Parámetros

Se establece un rango de valores posibles para que el algoritmo explore de forma automática.



Exploración Aleatoria y Eficiente

A diferencia de GridSearch, prueba combinaciones al azar, lo que permite explorar espacios grandes más rápido y con menor coste computacional.

Evaluación por Validación Cruzada

Cada combinación se pone a prueba rigurosamente para identificar cuál ofrece el mejor rendimiento real.

Fase 3: Documentación Automática con MLflow Autolog

MLflow como Bitácora Digital

Plataforma diseñada para registrar qué experimentos se iniciaron, con qué configuración y qué resultados se obtuvieron.



Activación de mlflow.sklearn.autolog()

Al activar este comando, MLflow detecta el entrenamiento y registra automáticamente parámetros, métricas y el mejor modelo encontrado.



Adiós a los Notebooks Desordenados

Se elimina la dependencia de "print" manual; ahora puedes comparar búsquedas entre el de forma visual y estructurada.



Conclusión: El Salto Profesional



De "Prueba y Error" a Trazabilidad Total

"Este modelo mejoró cuando ajustamos estos parámetros, y aquí están los resultados".



El Valor de la Reproducibilidad

La capacidad de volver atrás y entender por qué un modelo era mejor que otro es lo que define el Machine Learning profesional.

Criterios de Selección del Mejor Modelo

Métricas técnicas

- Clasificación: AUC-ROC, F1-score, Precision-Recall según contexto.
- Regresión: RMSE, MAE, MAPE.
- Ranking: NDCG, MAP.

Criterios de negocio

- Latencia de inferencia (SLA): ¿cuánto tarda en predecir?
- Tamaño del modelo: restricciones de memoria en dispositivos edge.
- Interpretabilidad: sectores regulados (finanzas, salud).

Model Selection Guide – Scikit-learn: https://scikit-learn.org/stable/model_selection.html



Registro del Mejor Modelo en MLflow Model Registry



Flujo de registro

- 1. Identificar el run con la mejor métrica objetivo.
- 2. Registrar: `mlflow.register_model(model_uri, 'NombreModelo')`.
- 3. Transición de etapa: Staging → Production → Archived.
- 4. Añadir descripción y tags al modelo registrado.

Selección programática

```
client = MlflowClient() | runs = client.search_runs(exp_id,  
order_by=['metrics.val_auc DESC']) | best_run = runs[0]
```

MLflow Model Registry: <https://mlflow.org/docs/latest/model-registry.html>

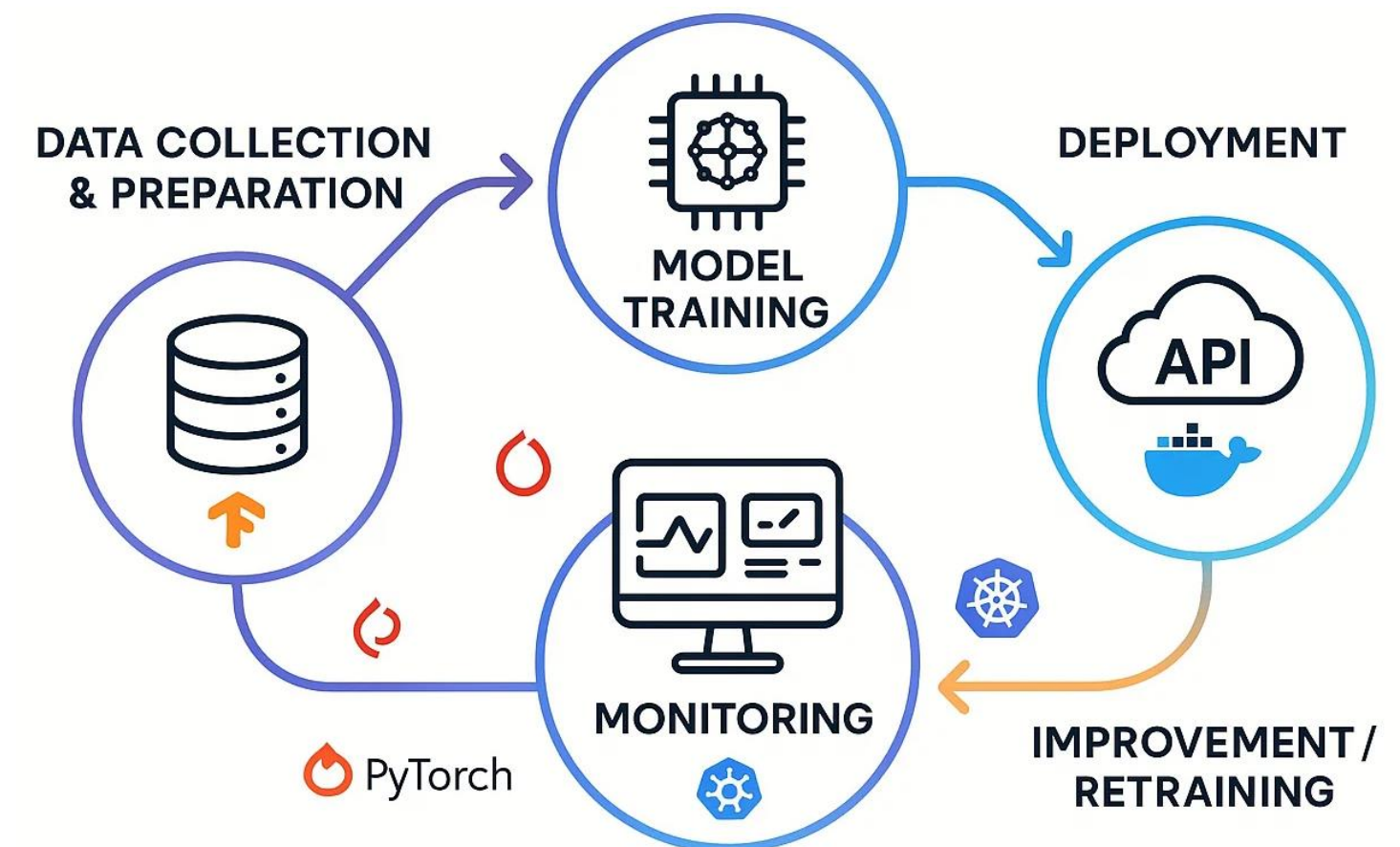
Pipeline Completo de Tuning y Selección

Flujo end-to-end en MLOps

- 1. Definir espacio de búsqueda y estrategia (Optuna/RandomSearch).
- 2. Ejecutar N trials; cada trial = 1 run en MLflow.
- 3. Comparar runs en MLflow UI (parallel coords, tabla).
- 4. Seleccionar el mejor run por métrica + criterios de negocio.
- 5. Registrar el modelo en MLflow Model Registry.
- 6. Promover a Staging → validación QA → Production.

MLflow End-to-End Example: <https://mlflow.org/docs/latest/getting-started/quickstart-2/index.html>

Guía MLOps Maturity Model: <https://docs.microsoft.com/azure/architecture/example-scenario/mlops/mlops-maturity-model>



Resumen, Referencias y Próximos Pasos

Lo que aprendimos hoy

- Hiperparámetros: qué son, cómo definen el espacio de búsqueda.
- Grid vs. Random vs. Bayesiano vs. Hyperband: cuándo usar cada uno.
- MLflow Tracking: logging reproducible, runs comparables.
- Selección del mejor modelo con criterios técnicos y de negocio.

Referencias principales

MLflow Docs: <https://mlflow.org/docs/latest/index.html>

Optuna: <https://optuna.readthedocs.io>

Scikit-learn Model Selection: https://scikit-learn.org/stable/model_selection.html

Practical MLOps (O'Reilly): <https://www.oreilly.com/library/view/practical-mlops/9781098103002/>

Full Stack Deep Learning – Hyperparameter Tuning: <https://fullstackdeeplearning.com/course/2022/lecture-6-hyperparameter-tuning/>